

# LED Lab 4

Welcome to the fourth lab in our series on Light-Emitting Diodes (LEDs)!

Building on the non-blocking timing concepts from Lab 3, this project introduces parallel execution. In the previous lab, our LEDs were locked into a single alternating pattern. In this lab, we treat the two pairs of LEDs as separate tasks. By using multiple timer variables, we can control them independently, even though they are currently programmed to blink in unison.

The primary objective of this lab is for students to:

- **Manage multiple time-based events simultaneously using separate previousMillis variables.**
- **Understand the concept of parallel multitasking in a single-threaded microcontroller.**
- **Reinforce the use of Boolean logic to toggle LED states.**

## Lab 4 – 4-Way Independent LED Blinker

In this lab, we have designed our code to handle two distinct groups of LEDs. Instead of one timer controlling everything, we establish two separate time-tracks. This structure allows the Arduino to check if it's time to blink Pair 1, and then immediately check if it's time to blink Pair 2, without one waiting for the other.

### Enter the Code

Open your Arduino IDE and type in the code below.

```
//Define the LED pins in the code

int LED_1 = 2;  //LEDs 1 and 2 will blink together

int LED_2 = 4;

int LED_3 = 6;  //LEDs 3 and 4 will blink together

int LED_4 = 8;


// Variables to track timing without blocking

unsigned long previousMillis1 = 0;

unsigned long previousMillis3 = 0;


// Blink intervals (milliseconds)

const long interval1and2 = 500; // LEDs 1&2 blink every 500ms
```

```
const long interval3and4 = 500; // LEDs 3&4 blink every 500ms
```

```
// LED states
```

```
bool state1and2 = false;
```

```
bool state3and4 = false;
```

```
void setup() {
```

```
    // Set LED pins as outputs
```

```
    pinMode(LED_1, OUTPUT);
```

```
    pinMode(LED_2, OUTPUT);
```

```
    pinMode(LED_3, OUTPUT);
```

```
    pinMode(LED_4, OUTPUT);
```

```
}
```

```
void loop() {
```

```
    unsigned long currentMillis = millis();
```

```
    // Handle LEDs 1&2 (blink together)
```

```
    if (currentMillis - previousMillis1 >= interval1and2) {
```

```
        previousMillis1 = currentMillis;
```

```
        state1and2 = !state1and2;
```

```
        digitalWrite(LED_1, state1and2);
```

```
        digitalWrite(LED_2, state1and2);
```

```
    }
```

```
    // Handle LEDs 3&4 (blink together)
```

```
    if (currentMillis - previousMillis3 >= interval3and4) {
```

```
        previousMillis3 = currentMillis;
```

```
        state3and4 = !state3and4;
```

```
        digitalWrite(LED_3, state3and4);
```

```
        digitalWrite(LED_4, state3and4);
```

```
    }
```

```
}
```

### Mini Lab: 4-Way Independent LED Blinker (Code Overview)

```
int LED_1 = 2;
```

```
int LED_2 = 4;
```

```
int LED_3 = 6;
```

```
int LED_4 = 8;
```

These lines define which digital pins on the Arduino are connected to each of the four LEDs. We are grouping them into two pairs: LEDs 1 & 2, and LEDs 3 & 4.

```
unsigned long previousMillis1 = 0;
```

```
unsigned long previousMillis3 = 0;
```

Here is the key difference from Lab 3. Instead of one variable tracking time, we have two.

- `previousMillis1` tracks the last time the first pair (LEDs 1 & 2) blinked.
- `previousMillis3` tracks the last time the second pair (LEDs 3 & 4) blinked. This allows the two pairs to operate on completely different schedules if needed.

```
const long interval1and2 = 500;
```

```
const long interval3and4 = 500;
```

These constants determine the speed of the blink. Currently, both are set to 500ms (half a second), so the LEDs will blink together. However, because they are separate variables, you could easily change the value of `interval3and4` from 500 to something faster like 200 to make the second pair blink much faster than the first!

```
bool state1and2 = false;
```

```
bool state3and4 = false;
```

We use Boolean variables to track the current status (ON or OFF) of each pair. `false` corresponds to LOW (OFF), and `true` corresponds to HIGH (ON).

```
if (currentMillis - previousMillis1 >= interval1and2) {  
    previousMillis1 = currentMillis;  
    state1and2 = !state1and2;  
    digitalWrite(LED_1, state1and2);  
    digitalWrite(LED_2, state1and2);  
}
```

The Arduino checks if 500ms has passed specifically for the first pair. If yes, it updates the **previousMillis1** timer, toggles the state (ON to OFF, or OFF to ON), and writes that state to pins 2 and 4.

```
if (currentMillis - previousMillis3 >= interval3and4) {  
    previousMillis3 = currentMillis;  
    state3and4 = !state3and4;  
    digitalWrite(LED_3, state3and4);  
    digitalWrite(LED_4, state3and4);  
}  
}
```

After checking the first pair, the Arduino checks the second pair using its own timer (**previousMillis3**) and interval (**interval3and4**). This independence is what allows for complex, overlapping patterns in advanced projects.

### Mini Lab: 4-Way Independent LED Blinker (Hardware Overview)

Since this project uses the same four LEDs as the previous lab, the hardware setup remains identical. We still require four separate current-limiting resistors.

| Component        | Quantity | Key Function                                                     |
|------------------|----------|------------------------------------------------------------------|
| Arduino UNO R3   | 1        | The main programmable controller.                                |
| Breadboard       | 1        | Used to connect all components without soldering.                |
| 5mm LED          | 4        | Converts electrical energy into light.                           |
| 100-ohm Resistor | 4        | <b>Limits the current</b> to protect each LED from burning out.  |
| Jumper Wires     | 10       | Used to connect the four LEDs and resistors to the Arduino pins. |

### Connecting Everything (Step by Step Instructions)

**Place the LEDs:** Insert the four 5mm LEDs into the breadboard. Ensure the longer leg (Anode) is oriented correctly.

**Attach the Resistors:** Connect one lead of a 100  $\Omega$  resistor to the shorter leg (Cathode) of each LED.

**Connect to Ground:** Connect the free ends of all four resistors to the **GND** rail on the breadboard, and connect that rail to the GND pin on the Arduino.

**Connect to Digital Pins:** Use jumper wires to connect the longer leg (Anode) of each LED to the Arduino as follows:

- **LED 1:** Digital Pin 2
- **LED 2:** Digital Pin 4
- **LED 3:** Digital Pin 6
- **LED 4:** Digital Pin 8

**Upload and Observe:** Once the code is uploaded, all four LEDs should blink on and off. While they may look synchronized now, try changing interval3 and 4 to 1000 in the code to see them blink at different rates!